Application Endpoints

Scope assumptions

- Base URL: /api/v1
- APIs are CRUD + domain-specific operations
- DTO-driven APIs, transactional boundaries respected
- Soft validations, not UI-specific behaviour

Domain focus: Master data ownership (similar to insurer → branch hierarchy)

Agency APIs

- POST /agencies – Create agency
- GET /agencies – Get all agencies
- GET /agencies/{agencyId} – Get agency by ID
- PUT /agencies/{agencyId} – Update agency
- DELETE /agencies/{agencyId} – Deactivate agency

Agency Office APIs

- POST /agencies/{agencyId}/offices – Add office under agency
- GET /agencies/{agencyId}/offices – List offices by agency
- GET /offices/{officeId} – Get office details
- PUT /offices/{officeId} – Update office details
- DELETE /offices/{officeId} – Deactivate office

Address APIs (shared reference)

- POST /addresses
- GET /addresses/{addressId}

Domain focus: Asset & workforce management (similar to insured assets, agents)

Bus APIs

- POST /offices/{officeId}/buses – Register bus
- GET /offices/{officeId}/buses – List buses by office
- GET /buses/{busId} – Bus details
- PUT /buses/{busId} – Update bus
- DELETE /buses/{busId} – Retire bus

Driver APIs

- POST /offices/{officeId}/drivers – Register driver
- GET /offices/{officeId}/drivers – List drivers
- GET /drivers/{driverId} – Driver profile
- PUT /drivers/{driverId} – Update driver
- DELETE /drivers/{driverId} – Remove driver

Domain focus: Core operational logic (similar to policy plans & schedules)

Route APIs

- POST /routes – Create route
- GET /routes – Get all routes
- GET /routes/{routeId} – Route details
- PUT /routes/{routeId} – Modify route
- DELETE /routes/{routeId} – Disable route

Trip APIs

- POST /trips – Create trip schedule
- GET /trips – List all trips
- GET /trips/{tripId} – Trip details
- GET /trips/search?fromCity=&toCity=&date= – Search trips
- PUT /trips/{tripId} – Update trip
- PATCH /trips/{tripId}/close – Close trip (no more bookings)

Domain focus: Customer lifecycle & transactional flow

Customer APIs

- POST /customers – Register customer
- GET /customers/{customerId} – Customer profile
- PUT /customers/{customerId} – Update customer
- GET /customers – Customer list (admin)

Booking APIs

- POST /trips/{tripId}/bookings – Book seat
- GET /customers/{customerId}/bookings – Customer bookings
- GET /bookings/{bookingId} – Booking details
- PATCH /bookings/{bookingId}/cancel – Cancel booking

Seat Availability APIs

- GET /trips/{tripId}/seats – Seat availability
- GET /trips/{tripId}/seats/booked
- GET /trips/{tripId}/seats/available

Domain focus: Financial transactions & post-service feedback

Payment APIs

- POST /payments – Make payment
- GET /payments/{paymentId} – Payment details
- GET /customers/{customerId}/payments – Payment history
- GET /bookings/{bookingId}/payment – Booking payment info
- PATCH /payments/{paymentId}/status – Update payment status

Review APIs

- POST /trips/{tripId}/reviews – Submit review
- GET /trips/{tripId}/reviews – Trip reviews
- GET /customers/{customerId}/reviews – Customer reviews
- DELETE /reviews/{reviewId} – Remove review (admin/moderation)

High-Impact Reporting Endpoints

✔ 1. Trip Occupancy Report

Endpoint

GET /reports/trips/occupancy?tripDate=

Why these matters

- Measures how well seats are sold
- Helps decide:
  - Cancel trips
  - Increase frequency
  - Change pricing

Expected Response

JSON

```
[
{
"tripId": 1,
"route": "Mumbai-Pune",
"capacity": 50,
"bookedSeats": 45,
"availableSeats": 5,
"occupancyPercentage": 90
}
]
```

Concepts Tested

- Join: trips → buses → bookings
- Aggregation (COUNT, SUM)
- Derived logic (percentage calculation)

✔ **Interview Question:**
"How do you calculate occupancy if available_seats and bookings mismatch?"

✔ **2. Revenue by Agency Report**
Endpoint
GET /reports/revenue/by-agency?fromDate=&toDate=
Why these matters

- Shows which agency generates maximum revenue
- Used for:
    - Partner evaluation
    - Business expansion decisions

Expected Response
JSON
[
{
"agencyId": 1,
"agencyName": "Indian Travels",
"totalRevenue": 50000,
"totalBookings": 120
}
]

Required Join Chain
payments → bookings → trips → buses → offices → agencies

Concepts Tested

- Multi-table joins (5+ tables)
- Grouping (GROUP BY agency)
- Filtering (date range)
- Clean DTO mapping

✔ **Interview Question:**
"Which join type did you use and why?"

✔ **3. Frequent Customers Report**
Endpoint
GET /reports/customers/frequent?limit=10
Why these matters

- Identifies high-value customers
- Useful for:
    - Loyalty programs
    - Discounts
    - Retention strategies

Expected Response
JSON
[
{
"customerId": 101,
"name": "Aarav Sharma",
"totalBookings": 15,
"totalSpent": 7500
}
]

Concepts Tested

- Join: customers → payments → bookings
- Aggregation (COUNT + SUM)
- Sorting (ORDER BY totalSpent DESC)
- Limiting (TOP N)

✔ **Interview Question:**
"How would you optimize this query for large datasets?"

✔ **Why:**

Endpoint	Skill Level	What It Tests
Trip Occupancy	Medium	Business logic + aggregation
Revenue by Agency	High	Complex joins + real-world hierarchy
Frequent Customers	High	Analytics + optimization thinking